

Module 3: Data Link Layer

Complete Question & Answer Notes

Q1. Issues Handled by the Data Link Layer

The Data Link Layer (Layer 2 of the OSI model) sits between the Physical Layer and Network Layer. It deals with node-to-node (hop-to-hop) delivery of data. The main issues it handles are:

Core Responsibilities

1. **Framing:** The DLL breaks the raw bit stream from the Physical Layer into manageable units called *frames*. It adds header/trailer to mark the start and end of each frame.
2. **Physical Addressing:** It adds MAC (hardware) addresses to the frame header so the frame can reach the correct device on the same network segment.
3. **Flow Control:** Ensures the sender does not overwhelm a slow receiver by regulating the rate of data transmission.
4. **Error Control:** Detects and corrects errors that occur during physical transmission using techniques like parity, CRC, and ARQ protocols.
5. **Access Control:** When multiple devices share the same channel (e.g., Ethernet bus), the DLL determines which device has the right to transmit at a given time (MAC sub-layer).
6. **Synchronization:** Ensures sender and receiver are in sync so frames are recognized correctly.

Q2. Stop-and-Wait Flow Control

Idea: The sender sends *one frame* and then **stops and waits** for an acknowledgement (ACK) from the receiver before sending the next frame.

How it works:

1. Sender transmits Frame 0.
2. Sender stops and starts a timer.
3. Receiver gets Frame 0, sends ACK 1 (ready for frame 1).
4. Sender receives ACK, sends Frame 1.
5. If no ACK received within timeout → sender **retransmits**.

Efficiency:

$$\eta = \frac{T_t}{T_t + 2T_p}$$

where T_t = transmission time, T_p = propagation delay.

Drawback: Very inefficient when propagation delay is large the channel sits idle most of the time.

Q3. Sliding Window Flow Control

Idea: Instead of waiting for an ACK after each frame, the sender can have multiple frames “in flight” at once up to a **window size** W .

How it works:

- The sender maintains a *send window* a range of sequence numbers it is allowed to send without waiting for ACK.
- The receiver maintains a *receive window* sequence numbers it is ready to accept.
- As ACKs come in, the window “slides” forward, allowing new frames to be sent.

Efficiency:

$$\eta = \begin{cases} 1 & \text{if } W \geq 2a + 1 \\ \frac{W}{2a + 1} & \text{if } W < 2a + 1 \end{cases} \quad \text{where } a = \frac{T_p}{T_t}$$

Advantage: Keeps the channel busy much higher throughput than Stop-and-Wait.

Q4. Error Detection: Parity, CRC, and Checksum

4.1 Parity Check

Add 1 extra bit (parity bit) to make the total number of 1s **even** (even parity) or **odd** (odd parity).

Example – Even Parity

Data: 1011001 → count of 1s = 4 (already even) → parity bit = **0**

Transmitted: 10110010

If receiver gets 10110110 → count of 1s = 5 (odd) → **Error detected!**

Limitation: Cannot detect *even numbers* of bit errors (they cancel out).

4.2 Cyclic Redundancy Check (CRC)

CRC treats the data as a large binary number (polynomial) and divides it by a **generator polynomial**. The remainder is appended as the Frame Check Sequence (FCS).

1. Append r zeros to the data (where r = degree of generator).
2. Divide data by generator using **modulo-2 division** (XOR, no carry).
3. Remainder = CRC code; append it to original data.
4. Receiver divides received frame by same generator if remainder = 0, no error.

Very powerful detects all burst errors of length $\leq r$.

4.3 Checksum

Used in higher layers (TCP/UDP/IP).

1. Divide data into equal segments (e.g., 16-bit words).
2. Add all segments together using **1's complement arithmetic**.

3. Take 1's complement of the sum $\rightarrow$ this is the checksum.
4. Receiver adds all segments including checksum result should be all 1s.

Q5. Hamming Code

Hamming code adds **redundant (parity) bits** at positions that are powers of 2 (positions 1, 2, 4, 8, ...) to enable **single-bit error detection and correction**.

Rule: For m data bits, we need r parity bits such that:

$$2^r \geq m + r + 1$$

Example Encode data 1011

Step 1: $m = 4$ data bits. Find r : $2^3 = 8 \geq 4 + 3 + 1 = 8 \checkmark \Rightarrow r = 3$ parity bits.

Step 2: Total bits = $m + r = 7$. Positions: 1,2,3,4,5,6,7

Parity bits at positions: **1, 2, 4**

Data bits fill positions: **3, 5, 6, 7** $\leftarrow$ data 1,0,1,1

P_1	P_2	D_3	P_4	D_5	D_6	D_7
?	?	1	?	0	1	1

Step 3: Calculate parity bits (even parity):

- P_1 covers positions 1,3,5,7: bits = ?,1,0,1 $\Rightarrow P_1 = 0$ (make even)
- P_2 covers positions 2,3,6,7: bits = ?,1,1,1 $\Rightarrow P_2 = 1$
- P_4 covers positions 4,5,6,7: bits = ?,0,1,1 $\Rightarrow P_4 = 0$

Encoded word: 0 1 1 0 0 1 1

Error correction: If bit at position 5 flips to 1 $\rightarrow$ received: 0110111

Recalculate parity groups error syndrome = **5** $\rightarrow$ flip bit 5 back.

Q6. ARQ Protocols (Automatic Repeat reQuest)

ARQ combines flow control with error control by requesting retransmission of erroneous/lost frames.

6.1 Stop-and-Wait ARQ

- Send one frame, wait for ACK.
- If ACK not received (timeout) or NAK received $\rightarrow$ **retransmit same frame**.
- Uses sequence numbers 0 and 1 alternately to avoid duplicates.
- Simple but inefficient (same drawback as S&W flow control).

6.2 Go-Back-N ARQ

- Sender can transmit up to $W = 2^n - 1$ frames without ACK (window size).
- If frame i has an error: receiver **discards frame i and all subsequent frames**.
- Sender must **go back and retransmit frame i and all frames after it**.
- Receiver window size = 1 (it expects in-order delivery).

Example: Window = 4. Frames 0,1,2,3 sent. Frame 1 is lost. Sender retransmits frames 1, 2, 3 again.

6.3 Selective Reject (Selective Repeat) ARQ

- Only the **specific erroneous/lost frame** is retransmitted.
- Receiver **buffers out-of-order frames** and ACKs them individually.
- Window size: sender = 2^{n-1} , receiver = 2^{n-1} .
- More efficient than Go-Back-N but requires more receiver buffer memory.

Property	S&W	Go-Back-N	Sel. Reject
Sender window	1	$2^n - 1$	2^{n-1}
Receiver window	1	1	2^{n-1}
Retransmits	1 frame	From error onward	Only errored frame
Complexity	Low	Medium	High

Q7. HDLC (High-Level Data Link Control)

HDLC is a **bit-oriented** data link protocol developed by ISO. It is the basis for many modern protocols (PPP, LAP-B, etc.).

Station types:

- **Primary:** Controls the link, sends commands.
- **Secondary:** Responds to primary.
- **Combined:** Acts as both (used in balanced mode).

7.1 HDLC Frame Format

Flag	Address	Control	Data	FCS	Flag
8 bits	8+ bits	8/16 bits	Variable	16/32 bits	8 bits

- **Flag:** 01111110 marks start and end of frame.
- **Address:** Identifies the secondary station.
- **Control:** Identifies frame type I-frame (data), S-frame (supervisory), U-frame (unnumbered/management).
- **Data:** The payload (absent in S and some U frames).
- **FCS:** CRC-based error check (CRC-16 or CRC-32).

Bit stuffing is used: after every five consecutive 1s in the data, a 0 is inserted to avoid confusion with the flag pattern.

Q8. ALOHA and Slotted ALOHA

These are **random access** protocols for shared channels (no central coordinator).

8.1 Pure ALOHA

- Any station transmits whenever it has data **no timing restrictions**.
- If collision occurs (two stations transmit at the same time) → both wait a **random time** and retransmit.
- Vulnerable time** = $2T_t$ (a frame can collide with a frame starting anywhere in the $2T$ window).

$$\text{Max Throughput} = \frac{1}{2e} \approx \mathbf{18.4\%}$$

8.2 Slotted ALOHA

- Time is divided into fixed **slots** equal to one frame transmission time.
- Stations can only transmit at the **beginning of a slot**.
- This halves the vulnerable time to just T_t .
- Collision still possible but less likely.

$$\text{Max Throughput} = \frac{1}{e} \approx \mathbf{36.8\%}$$

	Pure ALOHA	Slotted ALOHA
Vulnerable period	$2T_t$	T_t
Max efficiency	18.4%	36.8%
Synchronization	Not needed	Required

Q9. Collision-Free Protocols

These protocols ensure no two stations ever transmit simultaneously, eliminating collisions entirely.

9.1 Bit-Map Protocol (Reservation)

- A **contention slot** is reserved for each station (bitmap).
- If a station wants to transmit, it sets its bit in the contention slot.
- After scanning all bits, stations transmit in order.
- Overhead = N bits per round (one per station). Works well under heavy load.

9.2 Token Passing

- A special control packet called a **token** circulates around the network.
- Only the station **holding the token** may transmit.
- After transmitting, the station passes the token to the next station.
- Guarantees orderly, collision-free access. Used in Token Ring (IEEE 802.5).

9.3 Binary Countdown

- Each station has a binary address. When it wants to transmit, it broadcasts its address bit-by-bit (MSB first).
- If a station sees a higher-priority bit from another station, it **drops out**.
- Station with highest address wins and transmits.
- Channel efficiency = $\frac{d}{d+\log_2 N}$ where d = data bits, N = stations.

Q10. IEEE 802 Standards

10.1 IEEE 802.3 Ethernet (CSMA/CD)

IEEE 802.3

- Defines **wired Ethernet** using **CSMA/CD** (Carrier Sense Multiple Access with Collision Detection).
- Originally used a bus topology with coaxial cable; now uses star topology with twisted pair (Cat5/6).
- Frame size: 64 to 1518 bytes (minimum ensures collision detection).
- Speeds range from original 10 Mbps to 10 Gbps+ (Fast Ethernet, Gigabit Ethernet).
- **CSMA/CD**: Station listens before transmitting (Carrier Sense); if collision detected, it stops, sends jam signal, waits random time (exponential backoff), retransmits.

10.2 IEEE 802.4 Token Bus

IEEE 802.4

- Uses a **bus topology** but with **token passing** for access control (collision-free).
- Stations are arranged logically in a ring (by address) on top of a physical bus.
- The token is passed from one station to the next in logical order.
- Deterministic access predictable delay, suitable for **real-time industrial control**.
- Used in manufacturing automation (MAP Manufacturing Automation Protocol).
- Largely obsolete today; replaced by Ethernet and fieldbus systems.

10.3 IEEE 802.5 Token Ring

IEEE 802.5

- Uses a **ring topology** with **token passing** access control.
- A token circulates around the ring; a station captures it to transmit.
- Supports **priority levels** higher-priority stations can reserve the token.
- Operates at 4 Mbps or 16 Mbps.
- **Monitors** detect and remove circulating frames and lost tokens.
- More deterministic than Ethernet, but physically fragile one break in the ring affects all.
- Now largely replaced by Fast/Gigabit Ethernet.

Feature	802.3	802.4	802.5
Topology	Bus/Star	Bus	Ring
Access Method	CSMA/CD	Token Bus	Token Ring
Deterministic	No	Yes	Yes
Speed	10M–10G+	1–10 Mbps	4/16 Mbps
Status	Widely used	Obsolete	Obsolete

End of Module 3 Notes Data Link Layer